Solutions: Day 34 Practice Problems (Hinglish)

Day 34 terminal challenges ke complete step-by-step commands aur in-depth technical explanations.

Solution 1: Production Server Triage & Path Resolution

1. Print Current Location

```
pwd
```

- **Explanation**: `pwd` ka matlab hota hai **Print Working Directory**. Yeh OS se puchta hai ki current shell process kis folder mein hai aur uska complete absolute path return karta hai (jaise `/home/engineer`).

2. Detailed Directory Listing (Including Hidden Dotfiles)

```
ls -la
# Ya alag alag flags:
ls -l -a
```

- **Explanation**:
  - `-l`: Long listing format show karta hai jisme permissions (`drwxr-xr-x`), owner, group, file size, aur last modified date-time dikhti hai.
  - `-a`: All files show karta hai jisme hidden files (jo dot `.` se shuru hoti hain jaise `.bashrc`, `.env`) bhi display hoti hain.

3. Absolute vs Relative Navigation

- **Path Classification**:
  - `/var/log/cv_pipeline/crash_dump.log` ek **Absolute Path** hai kyunki yeh root slash (`/`) se shuru hota hai. Chahe aap kisi bhi folder mein khade ho, absolute path hamesha root se shuru ho kar exact same file ko point karta hai.
- **Absolute Path se Log Folder mein jana**:

  ```
  cd /var/log/cv_pipeline
  ```

- **Relative Path se Do Level Upar `/var/` mein jana**:

```
cd ../..
```

- **Explanation**: `.` ka matlab current directory hota hai aur `..` ka matlab parent directory. `/var/log/cv_pipeline` se ek `..` kiya toh `/var/log` par pahunche, doosra `..` kiya toh seedhe `/var` par pahunch gaye bina `/var` explicitly type kiye.

4. Home Directory mein Wapas Jana

```
cd ~
# Ya simple:
cd
```

- **Explanation**: Tilde `~` symbol Bash shell mein directly current user ke home directory (`$HOME` jaise `/home/engineer`) ko represent karta hai. Sirf `cd` type karke Enter dabane se bhi shell seedhe home directory mein le jata hai.

Solution 2: MLOps Project Workspace Scaffolding

1. Directory Tree Creation

```
# Step 1: Root folder banao aur enter karo
mkdir sentiment_llm
cd sentiment_llm

# Step 2: Teen subdirectories banao
mkdir data src checkpoints

# Step 3: data ke andar raw aur processed banao
mkdir data/raw data/processed
```

Advanced one-liner command:

```
mkdir -p sentiment_llm/{src,checkpoints,data/{raw,processed}}
cd sentiment_llm
```

2. File Initialization via `touch`

```
# src folder ke andar files banao
touch src/dataset.py src/model.py src/train.py
```

```
# Root folder mein requirements aur hidden .env banao
touch requirements.txt .env
```

- **Explanation**: `touch` command bina file open kiye instantly 0 bytes ki empty file create kar deta hai. Ek sath multiple files ke naam doge toh saari files ek single command mein ban jati hain.

3. Timestamp Refresh

```
touch requirements.txt
```

- **Explanation**: Agar file pehle se exist karti hai, toh `touch` uske content ko bilkul touch nahi karta, balki uski Access Time (`atime`) aur Modification Time (`mtime`) ko current system clock par update kar deta hai. Build systems aur automated scripts isi timestamp ko dekh kar decide karte hain ki file nayi hai ya purani.

4. Verification Command

```
ls -la
ls -la src/
ls -la data/
```

- **Output Inspection**: Saari files aur folders proper permissions aur 0-byte sizes ke saath list ho jayengi.

Solution 3: Defensive Disk Cleanup & Safe Deletion Governance

1. Empty vs Non-Empty Directory Deletion

- **Empty Folder ko Delete Karna**:

  ```
  rmdir stale_cache
  ```

  - **Behavior**: Agar `stale_cache` bilkul empty hai toh delete ho jayega. Lekin agar usme galti se ek bhi file ya hidden file hui, toh `rmdir` turant fail ho jayega aur bolega: `rmdir: failed to remove 'stale_cache': Directory not empty` Isliye empty folders ke liye `rmdir` sabse safe command hai.
- **Non-Empty Folder Tree ko Delete Karna**:

  ```
  rm -rf run_epoch_01
  ```

- **Behavior**: `-r` (recursive) folder ke andar ke saare files aur sub-folders ko delete karta hai. `-f` (force) bina confirm puche silent deletion karta hai.

2. Dangerous Commands ka Risk Analysis

- **Space Wali Bhayanak Galti**:

```
rm -rf / tmp/old_checkpoints
```

  - **Kyun Tabahi Machi?**: `/` aur `tmp` ke beech galti se space aane ki wajah se Bash ne ise do alag arguments samjha:
    1. Pehla argument: `/` (Pure computer ka root directory!)
    2. Doosra argument: `tmp/old_checkpoints`
  - **Result**: Command pure operating system ke system binaries, kernels, libraries, aur user data ko root level se permanently delete karna shuru kar degi. Pura server crash ho jayega aur restart bhi nahi hoga.
- **Recycle Bin vs Terminal Deletion**:
  - Desktop GUI par jab aap file delete karte ho toh OS use Recycle Bin / Trash folder mein move karta hai jaha se Undo ho sakta hai.
  - Terminal mein `rm` command seedhe OS ka `unlink()` system call invoke karta hai. File ke hard links aur inode pointers instantly destroy ho jate hain aur disk blocks free mark ho jate hain. Koi Recycle Bin nahi hota, isliye terminal deletion permanent aur irreversible hoti hai!

3. Safe Deletion Workflow Protocol

Production machine par recursive `rm` chalane se pehle yeh 3 steps follow karein:

1. **Check Directory**: Hamesha pehle `pwd` chala kar confirm karo ki sahi directory mein khade ho.
2. **List First**: `rm` chalane se pehle wahi path `ls` ke saath chala kar dekho ki kaunsi files match ho rahi hain:

```
ls run_epoch_*
```

3. **Interactive Flag ya Specific Paths**: Jahan doubt ho wahan `-i` flag use karo taaki delete hone se pehle confirmation maange:

```
rm -ri run_epoch_01/
```\n